



CHROMAURA
DOCUMENTATION TECHNIQUE

ChromAura

2026

Promotion 2027

RÉDIGÉ PAR

Akram Ridou, Andrea Izzillo, César Capion, Gabin Clerbout, Jules Caron,
Klervi Choblet, Lucas Estrade & Thomas Graveline-Mercier

TABLE DES MATIÈRES

1	<i>IDÉE GÉNÉRALE</i>	2
2	<i>MODULES LOGICIELS</i>	2
3	<i>INSTALLATION</i>	4
4	<i>UTILISATION</i>	4
5	<i>CHOIX TECHNIQUES</i>	5
5.1	<i>Stack technique</i>	5
6	<i>PROBLÈMES RENCONTRÉS, PIVOTS, AJUSTEMENTS</i>	6

1. IDÉE GÉNÉRALE

Installation interactive : une caméra de profondeur (Kinect) détecte des visiteurs et projette des corps de particules colorées. En pointant du doigt, chaque visiteur peut dessiner dans les airs avec une traînée de lumière persistante, ou faire un geste pistolet (index+majeur, ou index avec le pouce armé) pour tirer des particules. Un easter egg caché (pouce levé puis rabattu rapidement) déclenche le cri de Wilhelm. Fond légèrement animé, lucioles ambiantes, musique d'ambiance et effets sonores complètent l'expérience.

2. MODULES LOGICIELS

```
1 chrom-aura/
2 |-- main.tscn                                # Scène de rendu regroupant les différentes particules
3 |-- camera_script.gd                         # Loop: camera -> gestes -> particules -> audio -> UI
4 |
5 |-- addons/gd-depthcam/                     # [GDExtension] Capture Kinect (RGB + profondeur)
6 |-- addons/GDMP/                             # [GDExtension] Binding MediaPipe
7 |
8 |-- hand_gesture_engine.gd                  # Reconnaissance des gestes :
9 |                                           # (index tendu, index+majeur tendus, pouce levé,
10 |                                          # pouce baissé, pistolet/finger gun)
11 |-- hand_pose.gd                            # Position lissée et geste actif d'une main suivie
12 |-- hand_observation.gd                    # Données brutes d'une main détectée sur une image
13 |-- gesture_detection.gd                   # Événement envoyé quand un geste démarre/change/finit
14 |-- hand_overlay.gd                        # Overlay debug pour les mains
15 |
16 |-- instructions.gd                         # Bandeau UI rotatif (gestes possibles + easter eggs)
17 |
18 |-- scripts/BodyDetector.cs                 # Détecte les corps depuis l'image de profondeur
19 |-- scripts/TrackedBody.cs                 # État d'un corps suivi (position, palette)
20 |-- scripts/MaskSample.cs                  # Alimente les particules (couleur = profondeur +
    palette)
21 |-- scripts/BodyPalette.cs                 # Les 5 palettes de couleurs disponibles
22 |-- scripts/particles.cs                   # Rendu des particules : silhouette du corps
23 |                                          # et traînée de dessin
24 |-- scripts/GunParticleManager.cs          # Tirs de particules pour le geste pistolet
25 |                                          # (projectiles, étincelles, muzzle flare)
26 |-- scripts/AmbientFireflies.cs           # Nuage persistant de lucioles ambiantes,
27 |                                          # repoussées par les corps détectés
28 |-- scripts/BodyDebugOverlay.cs            # Overlay debug (boîtes autour des corps)
29 |
30 |-- audio_manager.gd                       # Musiques d'ambiance + SFX (glitter, tir, cri de
    Wilhelm)
31 |-- shaders/background_waves.gdshader      # Fond animé
32 '-- tests/test_hand_gesture_engine.gd      # Tests du moteur de gestes
33 tests/test_audio_easter_egg.gd             # Tests du cri de Wilhelm (audio)
34 tests/test_camera_easter_egg.gd            # Tests du déclenchement de l'easter egg (caméra)
```

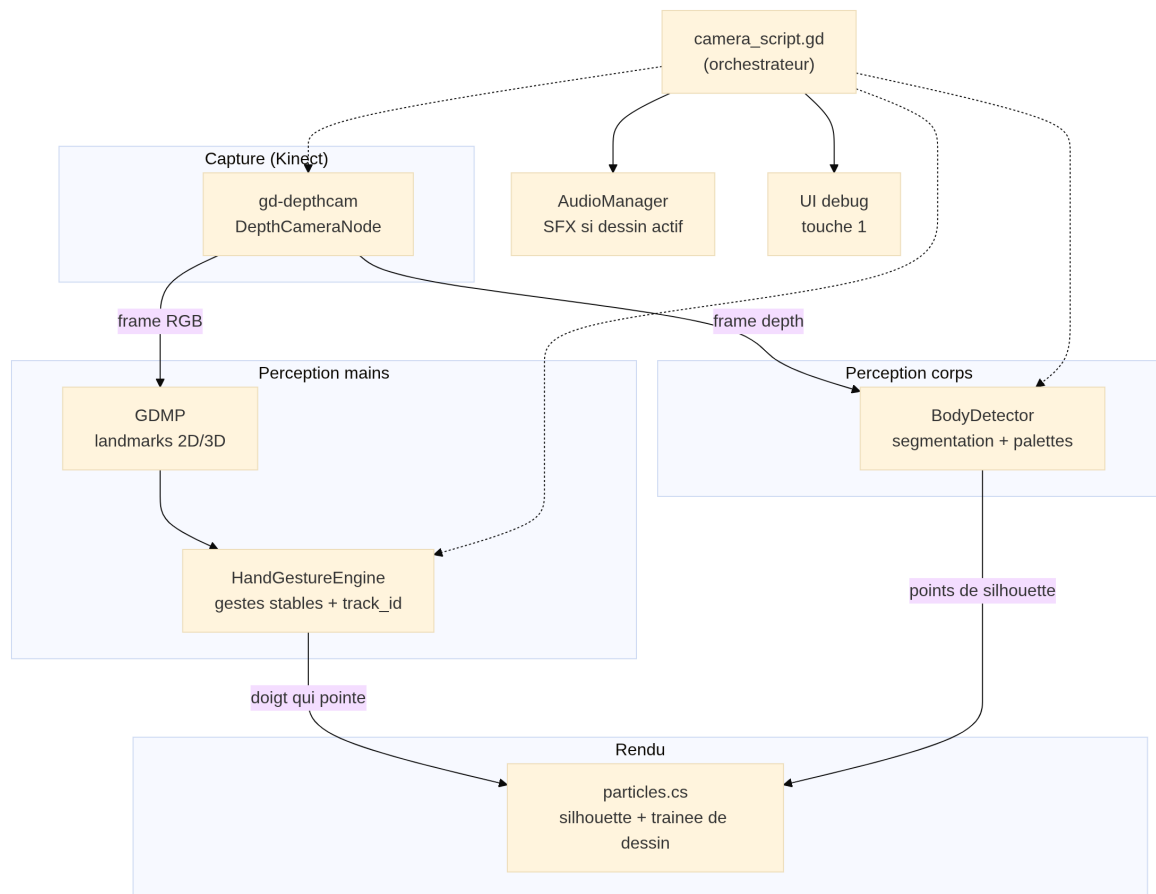


FIGURE 1 – Pipeline Godot détaillée.

3. *INSTALLATION*

- **Prérequis** : Godot 4.7.2 édition .NET/Mono (nécessaire pour le C#), et le .NET SDK 8.0. Un capteur Kinect v1 doit être branché avant le lancement (sinon le message “Kinect started (MediaPipe failed)” ou une absence de flux vidéo s’affichera).
- **Cloner le dépôt** : les binaires précompilés des GDExtensions (addons/gd-depthcam/bin) et les modèles MediaPipe (assets/models/hand_landmarker.task) sont déjà versionnés, aucun téléchargement supplémentaire n’est nécessaire.
- **Ouvrir le projet** : lancer Godot, ouvrir le dossier chrom-aura/. Au premier import, vérifier dans Project > Project Settings > Plugins que GDMP et gd-depthcam sont bien activés (normalement déjà déclarés dans project.godot).
- Si le binaire natif manque pour votre OS/architecture, un message dans la console Godot donne la commande SCons à lancer dans addons/gd-depthcam, puis redémarrer l’éditeur.
- **Lancer** : F5 dans l’éditeur, ou exporter un binaire. main.tscn démarre automatiquement le flux Kinect (auto_start).

4. *UTILISATION*

- Se placer face au capteur Kinect, dans son champ de profondeur (2 x 2) ; un corps détecté reçoit automatiquement une palette de couleur (parmi 5, attribuée aléatoirement).
- **Dessiner** : index tendu seul → une traînée de lumière persistante suit le bout du doigt.
- **Pistolet** : index + majeur tendus, ou index tendu avec le pouce armé vers le haut → tir de particules à cadence automatique (son de tir compris).
- **Cœur** : former un cœur avec les 2 mains → une multitude de cœurs apparaît à l’écran.
- **Easter egg “Wilhelm Scream”** : pouce levé puis rabattu rapidement (entre 0.2 et 3 secondes après le pouce levé) → cri de Wilhelm + explosion de particules à l’endroit de la main.
- Un bandeau en haut à gauche de l’écran alterne automatiquement (toutes les 3 secondes) entre les instructions de gestes disponibles, avec une chance d’environ 30% de montrer un indice caché sur un easter egg.
- **Touche 1** : bascule le mode développeur (squelette de main, bounding boxes des corps, flux RGB brut, statut des gestes actifs) — désactivé par défaut pour le public.

5. CHOIX TECHNIQUES

- **Kinect plutôt qu’une webcam classique** : le flux de profondeur natif permet une segmentation de corps fiable (silhouette, distance) sans modèle ML supplémentaire, indispensable pour l’effet d’aura par corps et le dégradé de couleur selon la profondeur.
- **Split GDScript / C#** : GDScript pour tout ce qui est événementiel et peu coûteux (gestes, audio, UI), C# pour ce qui doit tourner vite sur toute l’image (segmentation + particules).
- **Gestes reconnus géométriquement (pas par ML)** : la classification (`HandGestureEngine.classify_pose`) utilise uniquement des angles/distances normalisés dans le repère de la main (invariant en rotation et en échelle), pas de modèle entraîné. Plus simple à débbugger et à ajuster pendant la semaine qu’un classifieur ML.
- **Hystérésis sur les gestes** (seuils d’activation/maintien + délais) pour éviter le clignotement d’un geste (main qui tremble, frame ratée MediaPipe).
- **track_id stable par main** via un coût d’association (distance de centre de paume + écart d’échelle + pénalité de handedness) plutôt qu’un simple nearest-neighbor par frame, cela est nécessaire pour garder la même traînée de dessin attachée à la bonne main quand plusieurs personnes sont face à la caméra.

5.1. Stack technique

- **Moteur** : Godot 4.7, mode Forward+.
- **Logique de gameplay / UI** : GDScript (gestes, orchestration de la scène, audio). Choisi pour la facilité d’itération rapide et intuitive.
- **Traitement image intensif** : C# (.csproj, chrom-aura/scripts/) pour la segmentation de corps et le rendu de particules : les boucles pixel par pixel sur l’image de profondeur à chaque frame sont trop lourdes et pas assez modulables en GDScript pur.
- **Capture caméra** : Addon GodotDepthCamera (addons/gd-depthcam, compilé via SCons, binaires pré-compilés livrés dans bin/ pour Linux/Windows/macOS) exposant un nœud `DepthCameraNode` (streams RGB + profondeur du Kinect) qui permet de facilement récupérer les flux de profondeur et de couleur en GDScript.
- **Détection de mains** : Addon GDMP (chrom-aura/addons/GDMP), intégration de MediaPipe en GDScript, pour l’extraction des 21 landmarks de main. Tout le reste (classification de gestes, tracking multi-main, lissage temporel) est géré en GDScript, MediaPipe ne fournit que les points, pas les gestes.

6. *PROBLÈMES RENCONTRÉS, PIVOTS, AJUSTEMENTS*

- [Pivot] Webcam vs Kinect : un toggle Camera Source (WEBCAM/KINECT) a existé pour développer sans le capteur Kinect sous la main, pour utiliser MediaPipe seul.
- [Pivot] Moteur de gestes : abandon des gestes prédéfinis fournis par MediaPipe au profit d'un moteur maison basé sur les landmarks bruts + classification géométrique, pour garder le contrôle sur les gestes custom et pouvoir ajuster les seuils.
- [Problème → Ajustement] Fond animé : le shader de fond était trop présent et nuisait à la lisibilité des corps/particules ; intensité des vagues baissée à plusieurs reprises (intensité 0.55 → 0.1, nombre de bulles 10 → 1).
- [Ajustement] Couleur des corps : passage d'une attribution de palette déterministe (ordre fixe) à une attribution aléatoire parmi les palettes libres, pour varier les couleurs quand des visiteurs arrivent/partent.
- [Ajustements → Pivot] Geste de dessin : évolution en plusieurs étapes : seuil du doigt levé abaissé, persistance du geste de pointage, avant une refonte complète en système multi-main avec hystérésis.
- [Problème → Ajustement] Émission au bout du doigt : la traînée de dessin, d'abord émise sur toute la main (imprécis), a été isolée pour ne partir que du bout du doigt détecté.